# 1η Εργαστηριακή Άσκηση

Υλοποίηση πολυνηματικής λειτουργίας σε μηχανή αποθήκευσης δεδομένων

**Πασχάλης Πιπίδης 3064**
**Χρήστος Γεωργίου Μουσσές 4206**

3 Απριλίου 2023

## Εισαγωγή

Η εργασία αυτή βασίστηκε στην περσινή μας προσπάθεια πάνω στην ίδια άσκηση και δόθηκε περισσότερη έμφαση στην βελτίωση του ταυτοχρονισμού και τη βελτίωση του προϋπάρχοντα κώδικα.

Για τη συγκεκριμένη άσκηση θα τροποποιήσουμε τη μηχανή αναζήτησης **KiWi** έτσι ώστε να υποστηρίζει πολυνηματική λειτουργία των εντολών put και get που ήδη μας παρέχει. Η υλοποίησή μας, δημιουργεί συγκεκριμένο αριθμό νημάτων που ορίζει ο χρήστης και τους επιτρέπει να καλούν τις εντολές put και get ταυτόχρονα, καθώς επίσης και να ορίζει το ποσοστό των νημάτων το οποίο εκτελεί την κάθε λειτουργία. Η μηχανή αποθήκευσης έχει επίσης τροποποιηθεί κατάλληλα, έτσι ώστε να εκτελεί τις λειτουργίες αυτές ταυτόχρονα και να διατηρεί στατιστικά του χρόνου εκτέλεσής τους.

## Επέκταση Γραμμής Εντολών

Για την επιτυχή εκτέλεση των αλλαγών που κάναμε στη μηχανή αποθήκευσης, επεκτείναμε τα ορίσματα που πρέπει να δίνονται στη γραμμή εντολών κατά την εκτέλεση του προγράμματος **kiwi-bench**. Πιο συγκεκριμένα, κρατήσαμε άθικτες τις αρχικές λειτουργίες σειριακής ανάγνωσης και εγγραφής και προσθέσαμε δύο επιπλέον λειτουργίες. Η πρώτη λειτουργία είναι η παράλληλη ανάγνωση από τη μηχανή και η δεύτερη είναι η παράλληλη ανάγνωση και εγγραφή με επιλογή ποσοστού εγγραφών και αναγνωσεων.

Για την εκτέλεση του προγράμματος, παραθέτουμε παρακάτω το usage:

```
Usage:
./kiwi-bench <read|write> <op_num> [random]
./kiwi-bench <p-read> <op_num> <thread_num> [random]
./kiwi-bench <p-rw> <op_num> <thread_num> <writer_perc> [random]

read:           serial read from database
write:          serial write to database
p-read:         parallel read from database
p-rw:           parallel read and write

op_num:         number of operations to execute
thread_num:     number of threads (>0) to execute op_num operations
writer_perc:    percentage of write operations in [0, 100] (p-rw mode only)
random:         if present, operation values are randomized
```

Για την επέκταση των ορισμάτων του προγράμματος, τροποποιήσαμε το αρχείο **bench.c** κατάλληλα έτσι ώστε να γίνονται οι απαραίτητοι έλεγχοι. Σε περίπτωση που τα ορίσματα δε δοθούν σωστά, εμφανίζεται το παραπάνω μήνυμα usage.

## Παράδειγμα εκτέλεσης του προγράμματος

Η τελική εντολή make που χρησιμοποιήσαμε κάνει make clean και έπειτα make all για να βεβαιωθούμε πως δεν υπάρχουν παλαιότερα δεδομένα στην βάση δεδομένων και παρακάτω παρατίθεται η ίδια η εντολή καθώς και τα αποτελέσματα της. Η εντολή χρησιμοποίθηκε εντός του φακέλου bench.

```
cd .. && make clean && make all && cd bench
```

Με το παρακάτω αποτέλεσμα.

```
myy601@myy601lab1:~/kiwi/kiwi-source/bench$ cd .. && make clean && make all &&
↪  cd bench
cd engine && make clean
make[1]: Entering directory '/home/myy601/kiwi/kiwi-source/engine'
rm -rf *.o libindexer.a
make[1]: Leaving directory '/home/myy601/kiwi/kiwi-source/engine'
cd bench && make clean
make[1]: Entering directory '/home/myy601/kiwi/kiwi-source/bench'
rm -f kiwi-bench
rm -rf testdb
make[1]: Leaving directory '/home/myy601/kiwi/kiwi-source/bench'
cd engine && make all
make[1]: Entering directory '/home/myy601/kiwi/kiwi-source/engine'
    CC db.o
    CC memtable.o
    CC indexer.o
    CC sst.o
    CC sst_builder.o
    CC sst_loader.o
    CC sst_block_builder.o
    CC hash.o
    CC bloom_builder.o
    CC merger.o
    CC compaction.o
    CC skiplist.o
    CC buffer.o
    CC arena.o
    CC utils.o
    CC crc32.o
    CC file.o
    CC heap.o
    CC vector.o
    CC log.o
    CC lru.o
    AR libindexer.a
make[1]: Leaving directory '/home/myy601/kiwi/kiwi-source/engine'
cd bench && make all
make[1]: Entering directory '/home/myy601/kiwi/kiwi-source/bench'
gcc -g -ggdb -Wall -Wno-implicit-function-declaration
↪  -Wno-unused-but-set-variable bench.c kiwi.c -L ../engine -lindexer
↪  -lpthread -lsnappy -o kiwi-bench
make[1]: Leaving directory '/home/myy601/kiwi/kiwi-source/bench'
```

Για να εκτελεστεί το πρόγραμμα σε πολυνηματικό read-write mode χρησιμοποιείται η εντολή **./kiwi-bench** εντός του φακέλου **bench** ως εξής

```
myy601@myy601lab1:~/kiwi/kiwi-source/bench$ ./kiwi-bench p-rw 100000 6 67
```

Όπου "p-rw" η επιλεγμένη λειτουργία (εδώ "parallel read and write"), "100000" ο αριθμός των συνολικών λειτουργιών που θα εκτελεστούν, "6" ο αριθμός των νημάτων που θα χρησιμοποιηθούν για όλες τις λειτουργίες και "67" το ποσοστό των λειτουργιών που θα είναι εγγραφές.

Παρακάτω βλέπουμε το αποτέλεσμα χωρίς τα διάφορα print που κάνει από μόνο του το **KiWi**, τα οποία κλείσαμε προσωρινά για αυτό το παράδειγμα, και τα παραδείγματα που ακολουθούν βάζοντας σε σχόλιο τη γραμμή

```
#define PRINT
```

στο αρχείο **indexer.h** του φακέλου **engine**. Τα μόνα προϋπάρχοντα print που είχαν παραμείνει ενεργά και δεν συμπεριλήφθηκαν στην παρούσα αναφορά ήταν τα print των στατιστικών και της ενημέρωσης της πρόοδου των συναρτήσεων **_write_test()** και **_read_test()** του αρχείου **kiwi.c**.

```
Total reader cost: 3.000000sec, Total reader cost per operation:
↪  0.000273sec/op, Total reader throughput: 33000.000000ops/sec
Average reader cost per thread: 1.000000sec, Average reader cost per operation:
↪  0.000091sec/op, Average reader thread throughput: 11000.000000ops/sec
Total writer cost: 3.000000sec, Total writer cost per operation:
↪  0.000134sec/op, Total writer throughput: 66999.000000ops/sec
Average writer cost per thread: 1.000000sec, Average writer cost per operation:
↪  0.000045sec/op, Average writer thread throughput: 22333.000000ops/sec
```

Έπειτα ξανατρέξαμε το πρόγραμμα για μερικές διαφορετικές τιμές εισόδου, τις οποίες παραθέτουμε παρακάτω σε ζεύγη μαζί με τα αποτελέσματα τους. Για τα παρακάτω τεστ χρειάστηκε να μεγαλώσουμε τη χωρητικότητα του σκληρού δίσκου λόγω του μεγάλου αριθμού δεδομένων που εγγράφονται στη βάση. Για να το πετύχουμε αυτό χρησιμοποιήσαμε την εφαρογή **GParted**.

```
myy601@myy601lab1:~/kiwi/kiwi-source/bench$ ./kiwi-bench p-rw 10000000 8 50
```

```
Total reader cost: 350.000000sec, Total reader cost per operation:
↪  0.000280sec/op, Total reader throughput: 57148.392096ops/sec
Average reader cost per thread: 87.500000sec, Average reader cost per
↪  operation: 0.000070sec/op, Average reader thread throughput:
↪  14287.098024ops/sec
Total writer cost: 335.000000sec, Total writer cost per operation:
↪  0.000268sec/op, Total writer throughput: 59711.689554ops/sec
Average writer cost per thread: 83.750000sec, Average writer cost per
↪  operation: 0.000067sec/op, Average writer thread throughput:
↪  14927.922388ops/sec
```

```
myy601@myy601lab1:~/kiwi/kiwi-source/bench$ ./kiwi-bench p-read 555666 4
↪  random_keys
```

```
Total reader cost: 36.000000sec, Total reader cost per operation:
↪  0.000065sec/op, Total reader throughput: 246962.666667ops/sec
Average reader cost per thread: 9.000000sec, Average reader cost per operation:
↪  0.000016sec/op, Average reader thread throughput: 61740.666667ops/sec
```

```
myy601@myy601lab1:~/kiwi/kiwi-source/bench$ ./kiwi-bench p-rw 100000000 8 50
```

```
Total reader cost: 4264.000000sec, Total reader cost per operation:
↪  0.000341sec/op, Total reader throughput: 47142.637683ops/sec
Average reader cost per thread: 1066.000000sec, Average reader cost per
↪  operation: 0.000085sec/op, Average reader thread throughput:
↪  11785.659421ops/sec
Total writer cost: 4363.000000sec, Total writer cost per operation:
↪  0.000349sec/op, Total writer throughput: 45842.644925ops/sec
Average writer cost per thread: 1090.750000sec, Average writer cost per
↪  operation: 0.000087sec/op, Average writer thread throughput:
↪  11460.661231ops/sec
```

## Εφαρμογή πολυνηματισμού

Μια πρώτη προσέγγιση στην εφαρμογή του πολυνηματισμού απαιτεί την ταυτόχρονη ύπαρξη αναγνωστών στη μηχανή. Γι' αυτό θα χρειαστεί η αλλαγή των συναρτήσεων **_read_test()** και **_write_test()** στο αρχείο **kiwi.c**. Επειδή επιλέξαμε τα νήματα να καλούν αυτές τις δύο συναρτήσεις, επεκτείνουμε τα ορίσματα των συναρτήσεων κατάλληλα έτσι ώστε να δέχονται τη βάση δεδομένων ως όρισμα καθώς θέλουμε να έχουμε μόνο ένα στιγμιότυπό της όταν γίνεται η παράλληλη ανάγνωση. Τη βάση, την ανοίγουμε ακριβώς πριν ξεκινήσουμε τις αναγώσεις ή τις εγγραφές, και την κλείνουμε αφότου τελειώσουν. Όσον αφορά οποιαδήποτε παράλληλη ενέργεια, δημιουργούμε αρχικά ένα καινούριο struct σε ένα καινουριο αρχείο **thread_args.h**. Στο ίδιο αρχείο, δηλώνουμε 2 κλειδαριές οι οποίες χρησιμοποιούνται για να εγγυηθούμε τη συνέπεια των μεταβλητών που αφορούν τα στατιστικά, όπως θα εξηγηθεί αναλυτικά παρακάτω στην ενότητα Καταγραφή Στατιστικών Απόδοσης.

```
#include "../engine/db.h"
#include <pthread.h>

typedef struct
{
    DB* db;
    long int count;
    int r;
} pthread_args;

pthread_mutex_t lock_w;
pthread_mutex_t lock_r;
```

Αυτό είναι απαραίτητο καθώς η συνάρτηση **pthread_create()** για να περάσει ορίσματα στη συνάρτηση την οποία καλεί το νήμα που δημιουργεί, χρειάζεται να τα περάσει "πακεταρισμένα". Αφότου κάνουμε include το αρχείο **thread_args.h** στο αρχείο **bench.h** είμαστε έτοιμοι να τροποποιήσουμε το αρχείο **bench.c** κατάλληλα έτσι ώστε να υλοποιήσουμε τις εντολές που θα επιτρέψουν στη βάση να υποστηρίζει ανάγνωση και εγγραφή από πολλά νήματα ταυτόχρονα.

Για την ύπαρξη πολλών ταυτόχρονων αναγνώσεων αρκεί να δημιουργήσουμε έναν πίνακα από νήματα, να "πακετάρουμε" τα ορίσματα που θέλουμε να περάσουμε και να εκτελέσει το καθένα τη συνάρτηση

```
void* parallel_read(void* args)
{
    pthread_args* pargs = (pthread_args*)args;
    _read_test(pargs->db, pargs->count, pargs->r);
    return NULL;
}
```

η οποία ορίζεται στο αρχείο **bench.c**. Έπειτα, για κάθε νήμα, καλούμε τη συνάρτηση **pthread_join()** έτσι ώστε το κάθε νήμα να περιμένει τα υπόλοιπα πριν τερματίσει το πρόγραμμα.

```
...
pthread_t threads[thread_count];
pthread_args args = {db, count, r};

for(int i = 0; i < thread_count; i++) pthread_create(&threads[i], NULL,
↪  parallel_read, (void*)&args);
for(int j = 0; j < thread_count; j++) pthread_join(threads[j], NULL);
...
```

Το επόμενο βήμα είναι να υλοποιήσουμε την υποστήριξη εγγραφών από πολλαπλά νήματα.

Εάν εφαρμόσουμε την παραπάνω στρατηγική, δημιουργείται πρόβλημα καθώς εάν καλέσουμε ταυτόχρονα πολλαπλές λειτουργίες put, δημιουργούνται segmentation faults και η δομή αποκτά ασυνεπή μορφή. Για την επίλυση του προβλήματος αυτού θα αναφερθούμε στην επόμενη ενότητα αναλυτικά. Προς το παρόν δείχνουμε πως επεκτείναμε το αρχείο **bench.c** με σκοπό το πρόγραμμα να υποστηρίζει πολλαπλούς αναγνώστες και γραφείς ταυτόχρονα.

Αρχικά ορίζουμε τη συνάρτηση

```
void* parallel_write(void* args)
{
    pthread_args* pargs = (pthread_args*)args;
    _write_test(pargs->db, pargs->count, pargs->r);
    return NULL;
}
```

έπειτα υπολογίζουμε τα πλήθη αναγνωστών και γραφέων με βάση το ποσοστό που ορίσαμε από τα ορίσματα της γραμμής εντολών. Για ευκολία χωρίζουμε τα νήματα κατα το ήμισυ με τα μισά να είναι νήματα γραφέις και τα υπόλοιπα μισά να είναι νήματα αναγνώστες. Έπειτα, υπολογίζουμε πόσες εγγραφές και αναγνώσεις έχουμε με βάση το ποσοστό που έδωσε ο χρήστης και μοιράζουμε ισάξια τη δουλειά μεταξύ των αναγνωστών και των γραφέων.

Για παράδειγμα, εάν ο χρήστης εκτελέσει το πρόγραμμα με την εντολή

```
myy601@myy601lab1:~/kiwi/kiwi-source/bench$ ./kiwi-bench p-rw 100000 6 67
```

θα δημιουργηθούν 3 νήματα αναγνωστών και 3 νήματα γραφέων. Τα νήματα γραφέων θα μοιραστούν 67.000 operations και τα νήματα αναγνωστών θα μοιραστούν τα υπόλοιπα 33.000.

Τέλος, ορίζουμε δύο πίνακες νημάτων, έναν για τους αναγνώστες και έναν για τους γραφείς, και όπως αναφέρθηκε προηγουμένως, "πακετάρουμε" τα ορίσματα που θέλουμε να περάσουμε δημιουργούμε τα νήματα, και για κάθε νήμα, καλούμε τη συνάρτηση **pthread_join()** έτσι ώστε το κάθε νήμα να περιμένει τα υπόλοιπα πριν τερματίσει το πρόγραμμα.

```
...
int writer_count = thread_count/2;
int reader_count = thread_count - writer_count;

float w_perc = (float)writer_perc/100;
int w_op = count*w_op/writer_count;
int r_op = count*(1.0f - w_op)/reader_count;


pthread_t writers[writer_count];
pthread_t readers[reader_count];
pthread_args rargs = {db, r_op, r};
pthread_args wargs = {db, w_op, r};

for(int i = 0; i < writer_count; i++) pthread_create(&writers[i], NULL,
↪  parallel_write, (void*)&wargs);
for(int j = 0; j < reader_count; j++) pthread_create(&readers[j], NULL,
↪  parallel_read, (void*)&rargs);
for(int k = 0; k < writer_count; k++) pthread_join(writers[k], NULL);
for(int l = 0; l < reader_count; l++) pthread_join(readers[l], NULL);
...
```

Μετά από όλη αυτή τη διαδικασία, είμαστε έτοιμοι να τροποποιήσουμε τη βάση μας κατάλληλα με σκοπό να μην έχουμε πρόβλημα στον συγχρονισμό των νημάτων αναγνωστών και γραφέων και να προσέξουμε να μην αφήσουμε τη βάση σε ασυνεπή μορφή.

## Πρόβλημα Αναγνωστών-Γραφέων

Όπως αναφέραμε παραπάνω, το πρόβλημα που πρέπει να λύσουμε κατά την υλοποίηση πολυνηματισμού με υποστήριξη πολλαπλών γραφέων έγκειται στο γεγονός πως αν αφήσουμε πολλούς γραφείς να αλλάξουν τη βάση, αυτή θα αποκτήσει ασυνεπή μορφή καθώς οι αναγνώστες αφενός δε θα είναι σίγουρο πως διαβάζουν επικαιροποιημένα δεδομένα και αφετέρου, θα δημιουργούνται segmentation faults.

Τη λύση σε αυτό το πρόβλημα τη βρίσκουμε κατά την επίλυση του προβλήματος αναγνωστών-γραφέων. Στη συγκεκριμένη περίπτωση, η κοινή δομή που έχουμε για να εφαρμόσουμε τη λύση είναι η βάση δεδομένων της μηχανής αναζήτησης.

Μια πρώτη και αφελής σκέψη που θα μπορούσε κάποιος να κάνει είναι να τοποθετήσει μια κλειδαριά και στο **db_add()** και μία στο **db_get()**. Δυστυχώς η σκέψη αυτή πάσχει καθώς οι αναγνώστες πρέπει να περιμένουν όσο κάποιος γρφέας γράφει, αλλά επίσης πρέπει να περιμένουν όσο κάποιος άλλος αναγνώστης διαβάζει. Το ίδιο ισχύει και για τους γραφείς.

Μια δεύτερη σκέψη θα ήταν να εισάχθούν μεταβλητές οι οποίες θα ήλεγχαν το εάν κάποιος διαβάζει ή γράφει μια δεδομένη χρονική στιγμη και εάν κάποιος κάνει κάποιο operation στη βάση τότε οι υπόλοιποι περιμένουν. Αυτή η σκέψη όμως, πάσχει από race condition καθώς υπάρχει πιθανότητα να μπουν ταυτόχρονα και ένας αναγνώστης και ένας γραφέας κάτι το οποίο δεν είναι επιθυμητό καθώς προκαλεί ασυνέπεια.

## Λύση

Στη λύση μας, πέρα από τις μεταβλητές ελέγχου ανάγνωσης και εγγραφής που αναφέραμε παραπάνω, εισήγαμε και μια μεταβλητή συνθήκης η οποία ειδοποιεί τα νήματα για το πότε τους επιτρέπεται (επειδή είναι ασφαλές) να γράψουν ή να διαβάσουν. Οι απαιτήσεις μας είναι πως μπορούν να διαβάζουν όσοι θέλουν, αλλά μόνο ένας επιτρέπεται να γράφει κάθε φορα.

Πιο συγκεκριμένα, όταν ένας αναγνώστης καλεί την **db_get()**, κλειδώνει μια κοινόχρηστη κλειδαριά και ελέγχει εάν υπάρχουν γραφείς οι οποίοι γράφουν εκείνη τη στιγμή. Εάν υπάρχει ενεργός γραφέας, περιμένει από τη μεταβλητή συνθήκης μέχρι αυτή να του επιτρέψει να συνεχίσει. Στη συνέχεια αυξάνει έναν μετρητή για να δηλώσει πως ξεκινάει την ανάγνωση, ξεκλειδώνει την κλειδαριά για τους επόμενους, και κάνει ανάγνωση από τη βάση. Αφότου τελειώσει την ανάγνωση, ξανακλειδώνει την κλειδαριά, μειώνει τον μετρητή δήλωσης ανάγνωσης και στέλνει σήμα εκπομπής στη μεταβλητή συνθήκης για να προχωρήσει με γραφείς (εάν αυτοί υπάρχουν) και ξεκλειδώνει την κλειδαριά. Η αλλαγή του κώδικα στο αρχείο **db.c** φαίνεται παρακάτω.

```
int db_get(DB* self, Variant* key, Variant* value)
{
    pthread_mutex_lock(&self->lock);
        while(self->writers) pthread_cond_wait(&self->cond, &self->lock);
        self->reading++;
    pthread_mutex_unlock(&self->lock);

    int sst_ret;
    if (memtable_get(self->memtable->list, key, value) == 1) sst_ret = 1;
    else sst_ret = sst_get(self->sst, key, value);

    pthread_mutex_lock(&self->lock);
        self->reading--;
        pthread_cond_broadcast(&self->cond);
    pthread_mutex_unlock(&self->lock);
    return sst_ret;
}
```

Όταν ένας γραφέας καλεί την **db_add()**, κλειδώνει μια κοινόχρηστη κλειδαριά, αυξάνει έναν μετρητή για να δηλώσει πως περιμένει για εγγραφή, και ελέγχει εάν υπάρχουν αναγνώστες ή γραφείς οι οποίοι διαβάζουν ή γράφουν εκείνη τη στιγμή. Εάν υπάρχει κάποιος που εργάζεται εκείνη τη στιγμή, περιμένει από τη μεταβλητή συνθήκης μέχρι αυτή να του επιτρέψει να συνεχίσει. Στη συνέχεια αυξάνει έναν μετρητή για να δηλώσει πως ξεκινάει την εγγραφή, ξεκλειδώνει την κλειδαριά για τους επόμενους, και γράφει στη βάση. Αφού τελειώσει την εγγραφή, ξανακλειδώνει την κλειδαριά, μειώνει τον μετρητή δήλωσης εγγραφής, μειώνει επίσης το μετρητή δήλωσης αναμονής εγγραφής, και στέλνει σήμα εκπομπής στη μεταβλητή συνθήκης για να προχωρήσει με άλλους αναγνωστες γραφείς (εάν αυτοί υπάρχουν) και ξεκλειδώνει την κλειδαριά. Η αλλαγή του κώδικα στο αρχείο **db.c** φαίνεται παρακάτω.

```c
int db_add(DB* self, Variant* key, Variant* value)
{
    pthread_mutex_lock(&self->lock);
        self->writers++;
        while(self->reading || self->writing) pthread_cond_wait(&self->cond,
↪   &self->lock);
        self->writing++;
    pthread_mutex_unlock(&self->lock);

    if (memtable_needs_compaction(self->memtable))
    {
        INFO("Starting compaction of the memtable after %d insertions and %d
↪   deletions",
             self->memtable->add_count, self->memtable->del_count);
        sst_merge(self->sst, self->memtable);
        memtable_reset(self->memtable);
    }

    int mem_ret = memtable_add(self->memtable, key, value);

    pthread_mutex_lock(&self->lock);
        self->writing--;
        self->writers--;
        pthread_cond_broadcast(&self->cond);
    pthread_mutex_unlock(&self->lock);
    return mem_ret;
}
```

Η παραπάνω λύση μας, επιλύει το δεύτερο πρόβλημα αναγνωστών γραφέων το οποίο δίνει προτίμηση στους γραφείς καθώς όπως εξηγήθηκε παραπάνω οι αναγνώστες περιμένουν τους γραφείς να τελειώσουν την εγγραφή πριν ξεκινήσουν να διαβάζουν, ενώ οι γραφείς περιμένουν είτε κάποιο γραφέα να τελειώσει να γράφει, είτε κάποιον αναγνώστη να σταματήσει να διαβάζει. Η λύση αυτή δεν είναι η αποδοτικότερη αλλά καταφέρνει να λύσει το πρόβλημά μας (N αναγνώστες 1 γραφέας). Επιπλέον η λύση μας αποφεύγει το starvation των γραφέων εάν υπάρχει μια συνεχής ροή αναγνωστών.

Η κλειδαριά, η μεταβλητή συνθήκης, και οι μετρητές που χρησιμοποιούνται στον κώδικά μας ορίζονται εντός του struct **_db** στο αρχείο **db.h**

```
typedef struct _db {
//    char basedir[MAX_FILENAME];
    char basedir[MAX_FILENAME+1];
    SST* sst;
    MemTable* memtable;

    int writers;
    int reading;
    int writing;

    pthread_mutex_t lock;
    pthread_cond_t cond;
} DB;
```

η αρχικοποίησή τους γίνεται στη συνάρτηση **db_open_ex()** στο αρχείο **db.c**

```
DB* db_open_ex(const char* basedir, uint64_t cache_size)
{
    DB* self = calloc(1, sizeof(DB));

    if (!self)
        PANIC("NULL allocation");

    self->reading = 0;
    self-> writing = 0;
    self->writers = 0;

    pthread_mutex_init(&self->lock, NULL);
    pthread_cond_init(&self->cond, NULL);

    strncpy(self->basedir, basedir, MAX_FILENAME);
    self->sst = sst_new(basedir, cache_size);

    Log* log = log_new(self->sst->basedir);
    self->memtable = memtable_new(log);

    return self;
}
```

### Δικαιοσύνη

Η παραπάνω λύση μας, δυστυχώς αν και λύνει το πρόβλημα των αναγνωστών-γραφέων επαρκώς, μπορεί να δημιουργήσει πρόβλημα στους αναγνώστες. Σε αντίθεση με τους γραφείς που αποφεύγουν το starvation, οι αναγνώστες πάσχουν από αυτό εάν υπάρχει μια συνεχής ροή γραφέων καθώς δίνεται προτεραιότητα σε αυτούς. Συνεπώς και το πρώτο αλλά και το δεύτερο πρόβλημα αναγνωστών-γραφέων πάσχουν από starvation κάτι το οποίο μπορεί να είναι επαρκές σε μια λύση (ανάλογα πάντα και με τη φύση του προβλήματος).

Ένα από τα μεγαλύτερα προβλήματα στον ταυτοχρονισμό νημάτων αποτελεί η δικαιοσύνη. Με τον όρο δικαιοσύνη αναφερόμαστε στην αρχή πως πολλαπλά νήματα τα οποία ανταγωνίζονται για έναν κοινόχρηστο πόρο έχουν ίση πιθανότητα να έχουν πρόσβαση σε αυτόν. Δυστυχώς δεν είναι πάντα εύκολο να υλοποιηθεί ένας τέτοιος μηχανισμός δικαιοσύνης καθώς υπάρχουν πολλοί παράγοντες που πρέπει κάποιος να λάβει υπ' όψη του. Για παράδειγμα, εάν για να εγγυηθούμε δικαιοσύνη περιορίσουμε πολύ την πρόσβαση στον πόρο μπορεί να έχουμε μειωμένη απόδοση.

Αφότου υλοποιήσαμε τη λύση του δευτέρου προβλήματος αναγνωστών-γραφέων αποφασίσαμε να προσπαθήσουμε να βρούμε μια πιο δίκαιη λύση με σκοπό να αποφύγουμε το starvation των αναγνωστών. Δυστυχώς δεν καταφέραμε να προχωρήσουμε σε κάποια υλοποίηση αλλά ενδεικτικά θα σας αναφέρουμε παρακάτω μια πιθανή αλλαγή της λογικής μας.

Μια πιθανή λύση στο πρόβλημά μας θα ήταν η χρήση μιας δομής FCFS (First Come First Served για παράδειγμα μια ουρά). Η λογική είναι πως τα νήματα που θέλουν πρόσβαση στη βάση εισέρχονται στην ουρά και έπειτα αυτή τους δίνει πρόσβαση στη βάση με τη σειρά που αυτά έφτασαν. Όταν ένα νήμα τελικά σταματήσει να "επεξεργάζεται" τον πόρο, αφαιρείται από την ουρά αφήνοντας τον επόμενο να έχει πρόσβαση. Με τη χρήση ουράς βεβαιωνόμαστε πως δε θα υπάρξει starvation είτε καθυστέρηση στην πρόσβαση του πόρου καθώς η επίτρεψη πρόσβασης των νημάτων στον πόρο με τη σειρά που έφτασαν εγγυάται δικαιοσύνη.

## Βαθύτερος Πολυνηματισμός

Αφιερώσαμε αρκετό χρόνο στην προσπάθεια να χρησιμοποιήσουμε πολυνηματισμό σε χαμηλότερα επίπεδα της μηχανής αποθήκευσης όπως το MemTable, το SkipList, το Sorted String Table (SST) και το Log του LSM-tree.

Πιο συγκεκριμένα, παρατηρήσαμε πως ήδη γίνεται merging των SkipLists σε SSTs σε ξεχωριστό thread όταν είναι defined το **BACKGROUND_MERGE** οπότε κάναμε μια προσπάθεια να βελτιώσουμε την διαδικασία, επιτρέποντας πολυνηματική πρόσβαση και επεξεργασία των SkipLists.
Για να το επιτύχουμε αυτό, χρησιμοποιήσαμε Concurrent SkipLists για να κάνουμε τα ήδη υπάρχοντα SkipLists thread safe χώρις να κλειδώνουμε ολόκληρο το SkipList κάθε φορά, αλλά μόνο τα SkipNodes που βρίσκονται ήδη υπό επεξεργασία από έναν γραφέα ώστε να μην δημιουργηθεί ασυνέπεια από άλλους γραφείς που θέλουν να επεξεργαστούν το ίδιο SkipNode.

Επιπλέον, η παραπάνω βελτίωση θεωρητικά θα βελτιώσει σε κάποιο βαθμό και την απόδοση της μηχανής όταν γίνονται εγγραφές και στο MemTable αφού και αυτό χρησιμοποιεί SkipLists για την εισαγώγη στοιχείων.

Δυστυχώς, δε καταφέραμε επιτυχώς την υλοποίηση των Concurrent SkipLists, διότι συχνά συναντούσαμε errors που αφορούσαν την κακή διαχείρηση μνήμης κατά την εκτέλεση του benchmark, το οποίο υποδηλώνει πως δεν ήταν thread safe η επέκταση μας στον αρχικό κώδικα του αρχείου **skiplist.c.**

```
Τα errors που συναντήσαμε:
realloc(): invalid next size
και
malloc(): corrupted top size1 adding key-1
```

Προχωρώντας στο MemTable, το μοναδικό σημείο που θεωρήσαμε καλό να υποστηρίξουμε πολυνηματικά ήταν η κλήση της συνάρτησης **skiplist_insert()** η οποία γίνεται εντός της συνάρτησης **_memtable_edit()** του αρχείου **memtable.c**.
Πιο συγκεκριμένα καλούσαμε σε ξεχωριστό thread την παραπάνω συνάρτηση και είχαμε ορίσει ως κρίσιμη περιοχή τον μετρητή εισαγωγών στο MemTable ώστε να μην δημιουργείται ασυνέπεια σε αυτή την μεταβλητή από τους πολλαπλούς γραφείς.

```
...
SkipList_Insert_Args* p_sli_args = {self->list, key->mem, key->length, opt,
↪   mem};
    pthread_create(self->insertion_thread, NULL, p_skiplist_insert,
↪   p_sli_args);
    void* p_sli_status;
    pthread_join(self->insertion_thread, &p_sli_status);
if ((int) p_sli_status == STATUS_OK_DEALLOC)
        free(mem);
if (opt == ADD){
        pthread_mutex_lock(&self->lock);
        self->add_count++;
        pthread_mutex_unlock(&self->lock);
...
```

Δυστυχώς, από τη στιγμή που αποτύχαμε να κάνουμε τα SkipLists thread safe η παρπάνω αλλαγή συχνά οδηγούσε σε ασυνέπεια στη βάση δεδομένων και άρα σε segmentation fault errors. Συνεπώς, δεν συμπεριλήφθηκε στον τελικό κώδικα που παραδώσαμε.

Τέλος, εξερευνήσαμε την δυνατότητα πολυνηματισμού των Logs του LSM-Tree, αλλά καταλήξαμε στο συμπέρασμα πως δοθέντων των ικανοτήτων, των γνώσεων αλλά και του χρόνου που μας απέμενε δεν ήταν ρεαλιστικά δυνατό να υλοποιήσουμε πολυνηματισμό και ταυτοχρονισμό της αποθήευσης και φόρτωσης των αρχείων που διαχειρίζεται το **log.c** και χωρίς να προκαλόυμε ασυνέπειες στα δεδομένα αλλά και χωρίς να υποβαθμίσουμε την αποδοτικότητα της μηχανής λόγω του αυξημένου φόρτου I/O λειτουργιών στο δίσκο.

## Καταγραφή Στατιστικών Απόδοσης

Για τον υπολογισμό και την καταγραφή των στατιστικών απόδοσης χρησιμοποιήσαμε μερικές global μεταβλητές για να καταγράφουμε το συνολικό κόστος, το κόστος ανά λειτουργία αλλά και το συνολικό throughput (διεκπεραιωτικότητα) για τους αναγνώστες και τους γραφείς ξεχωριστά. Τα παραπάνω τα χρησιμοποιούμε και για να υπολογίζουμε επιπλέον τον μέσο όρο κόστους ανά thread, τον μέσο όρο κόστους λειτουργίας ανά thread και το μέσο όρο throughput ανά thread, πάλι ξεχωριστά για αναγνώστες και γραφείς. Τέλος, τυπώνουμε στο τερματικό τα παραπάνω στατιστικά και τα αποθηκεύουμε σε ένα αρχείο CSV για περαιτέρω χρήση (π.χ. για δημιουργία διαγραμμάτων), μαζί με τις σχετικές μεταβλητές εισόδου της εντολής (π.χ. τύπος/οι λειτουργιάς/ών, αριθμός λειτουργιών, κλπ).
Πιο αναλυτικά, δηλώνουμε 6 global μεταβλητές στο αρχείο **bench.c** οι οποίες κρατάνε το συνολικό κόστος, το κόστος ανά λειτουργία αλλά και το συνολικό throughput (διεκπεραιωτικότητα) για τους αναγνώστες και τους γραφείς αντίστοιχα.

```
extern double readers_total_cost, readers_total_cost_per_op,
↪  readers_total_throughput;
extern double writers_total_cost, writers_total_cost_per_op,
↪  writers_total_throughput;
```

Έπειτα, στο αρχείο **kiwi.c** ορίζουμε τις αρχικές τιμές των μεταβλητών.

```
double readers_total_cost = 0.0f;
double readers_total_cost_per_op = 0.0f;
double readers_total_throughput = 0.0f;

double writers_total_cost = 0.0f;
double writers_total_cost_per_op = 0.0f;
double writers_total_throughput = 0.0f;
```

Στην συνέχεια επεκτείναμε τον αρχικό κώδικα εκτύπωσης στατιστικών στις συναρτήσεις **_write_test()** και **_read_test()** ώστε να υπολογίζονται και να αποθηκεύονται τα στατιστικά εντός κρίσιμης περιοχής, για να μην δημιουθργηθούν ασυνέπειες στις τιμές των μεταβλητών από τα πολλαπλά νήματα που τα χρησιμοποιούν.
**_write_test()**

```
...
pthread_mutex_lock(&lock_w);
cost = end -start;
writers_total_cost += cost;

printf(LINE);
printf("|Random-Write (done:%ld): %.6f sec/op; %.1f writes/sec(estimated);
↪  cost:%.3f(sec);\n"
    ,count, (double)(cost / count)
    ,(double)(count / cost)
    ,cost);
writers_total_cost_per_op += (double)(cost / count);
writers_total_throughput += (double)(count / cost);
pthread_mutex_unlock(&lock_w);
...
```

**_read_test()**

```
...
pthread_mutex_lock(&lock_r);
cost = end - start;
readers_total_cost += cost;

printf(LINE);
printf("|Random-Read (done:%ld, found:%d): %.6f sec/op; %.1f reads
↪  /sec(estimated); cost:%.3f(sec)\n",
    count, found,
    (double)(cost / count),
    (double)(count / cost),
    cost);
readers_total_cost_per_op += (double)(cost / count);
readers_total_throughput += (double)(count / cost);
pthread_mutex_unlock(&lock_r);
...
```

Έπειτα, για την εκτύπωση και αποθήκευση των αρχείων δημιούργουμε στην συνάρτηση **main()** του αρχείου **bench.c** ένα αρχείο με όνομα **stats.csv** εάν δεν υπάρχει ήδη, ελέγχουμε εάν το αρχείο δημιουργήθηκε και ανοίχθηκε επιτυχώς από το πρόγραμμα και αν όχι τυπώνουμε ένα σχετικό error message και τερματίζουμε την λειτουργία του προγράμματος. Αν το αρχείο δημιουργήθηκε και ανοίχθηκε επιτυχώς τότε ελέγχουμε αν είναι κενό ώστε αν είναι να βάλουμε στην πρώτη σειρά μια επικεφαλίδα για να επεξηγούμε ποιές πληροφορίες περιέχει η κάθε στήλη του αρχείου μας.

```
...
FILE *fstats = fopen("stats.csv", "a+");
if(fstats == NULL)
{
    fprintf(stderr, "Failed to create/open stats file, exiting");
    exit(1);
}
else
{
    int c = fgetc(fstats);
    if (c == EOF)
        fprintf(fstats, "OPERATION_TYPE, OP_COUNT, THREAD_COUNT,
↪  OP_TYPE_PERCENT, RANDOM_KEYS(Y/N), TOTAL_COST, AVG_COST_PER_THREAD,
↪  TOTAL_COST_PER_OP, AVG_COST_PER_OP, TOTAL_THROUGHPUT, AVG_THROUGHPUT\n");
}
...
```

Τέλος, υπολογίζουμε και τυπώνουμε στο τερματικό αλλά και στο αρχείο csv τα συνολικά στατιστικά από τις global μεταβλητές μας καθώς και τον μέσο όρο κόστους ανά thread, τον μέσο όρο κόστους λειτουργίας ανά thread και το μέσο όρο throughput ανά thread, για γραφείς και αναγνώστες ξεχωριστά με τον παρακάτω κώδικα σε περίπτωση test μόνο παράλληλων αναγνωστών

```
...
fprintf(stdout, "Total reader cost: %fsec, Total reader cost per operation:
↪  %fsec/op, Total reader throughput: %fops/sec\n",
               readers_total_cost, readers_total_cost_per_op,
↪  readers_total_throughput);
fprintf(stdout, "Average reader cost per thread: %fsec, Average reader cost per
↪  operation: %fsec/op, Average reader thread throughput: %fops/sec\n",
               readers_total_cost/(double)thread_count,
↪  readers_total_cost_per_op/(double)thread_count,
↪  readers_total_throughput/(double)thread_count);
fprintf(fstats, "%s, %ld, %d, 100, %d, %f, %f, %f, %f, %f, %f\n",
argv[1], count, thread_count, r, readers_total_cost,
↪  readers_total_cost/(double)thread_count, readers_total_cost_per_op,
↪  readers_total_cost_per_op/(double)thread_count, readers_total_throughput,
↪  readers_total_throughput/(double)thread_count);
...
```

και με τον παρακάτω κώδικα στην περίπτωση test παράλληλων αναγνωστών και γραφέων.

```
...
writer_perc = atoi(argv[4]);
int writer_count = thread_count/2;
int reader_count = thread_count - writer_count;

float w_perc = (float)writer_perc/100;

int w_op = (count * w_perc)/writer_count;
int r_op = (count *(1.0f - w_perc))/reader_count;
...
fprintf(stdout, "Total reader cost: %fsec, Total reader cost per operation:
↪  %fsec/op, Total reader throughput: %fops/sec\n",
                   readers_total_cost, readers_total_cost_per_op,
↪  readers_total_throughput);
    fprintf(stdout, "Average reader cost per thread: %fsec, Average reader cost
↪  per operation: %fsec/op, Average reader thread throughput: %fops/sec\n",
                   readers_total_cost/(double)reader_count,
↪  readers_total_cost_per_op/(double)reader_count,
↪  readers_total_throughput/(double)reader_count);
    fprintf(stdout, "Total writer cost: %fsec, Total writer cost per operation:
↪  %fsec/op, Total writer throughput: %fops/sec\n",
                   writers_total_cost, writers_total_cost_per_op,
↪  writers_total_throughput);
    fprintf(stdout, "Average writer cost per thread: %fsec, Average writer cost
↪  per operation: %fsec/op, Average writer thread throughput: %fops/sec\n",
```

```
                    writers_total_cost/(double)writer_count,
↪   writers_total_cost_per_op/(double)writer_count,
↪   writers_total_throughput/(double)writer_count);
    fprintf(fstats, "%s(r), %ld, %d, %d, %d, %f, %f, %f, %f, %f, %f\n",
    argv[1], count, reader_count, (100-writer_perc), r, readers_total_cost,
↪   readers_total_cost/(double)reader_count, readers_total_cost_per_op,
↪   readers_total_cost_per_op/(double)reader_count, readers_total_throughput,
↪   readers_total_throughput/(double)reader_count);
    fprintf(fstats, "%s(w), %ld, %d, %d, %d, %f, %f, %f, %f, %f, %f\n",
    argv[1], count, writer_count, writer_perc, r, writers_total_cost,
↪   writers_total_cost/(double)writer_count, writers_total_cost_per_op,
↪   writers_total_cost_per_op/(double)writer_count, writers_total_throughput,
↪   writers_total_throughput/(double)writer_count);
...
```

Για λόγους συνέπειας και πληρότητας κρατάμε και τυπώνουμε στατιστικά και για τις προυπάρχουσες σειριακές λειτουργίες, με μερικές hardcoded τιμές όπου αυτό είναι θεμιτό (π.χ. THREAD_COUNT = 1, OP_TYPE_PERCENT = 100, κλπ) με τον παρακάτω κώδικα για το σειριακό write

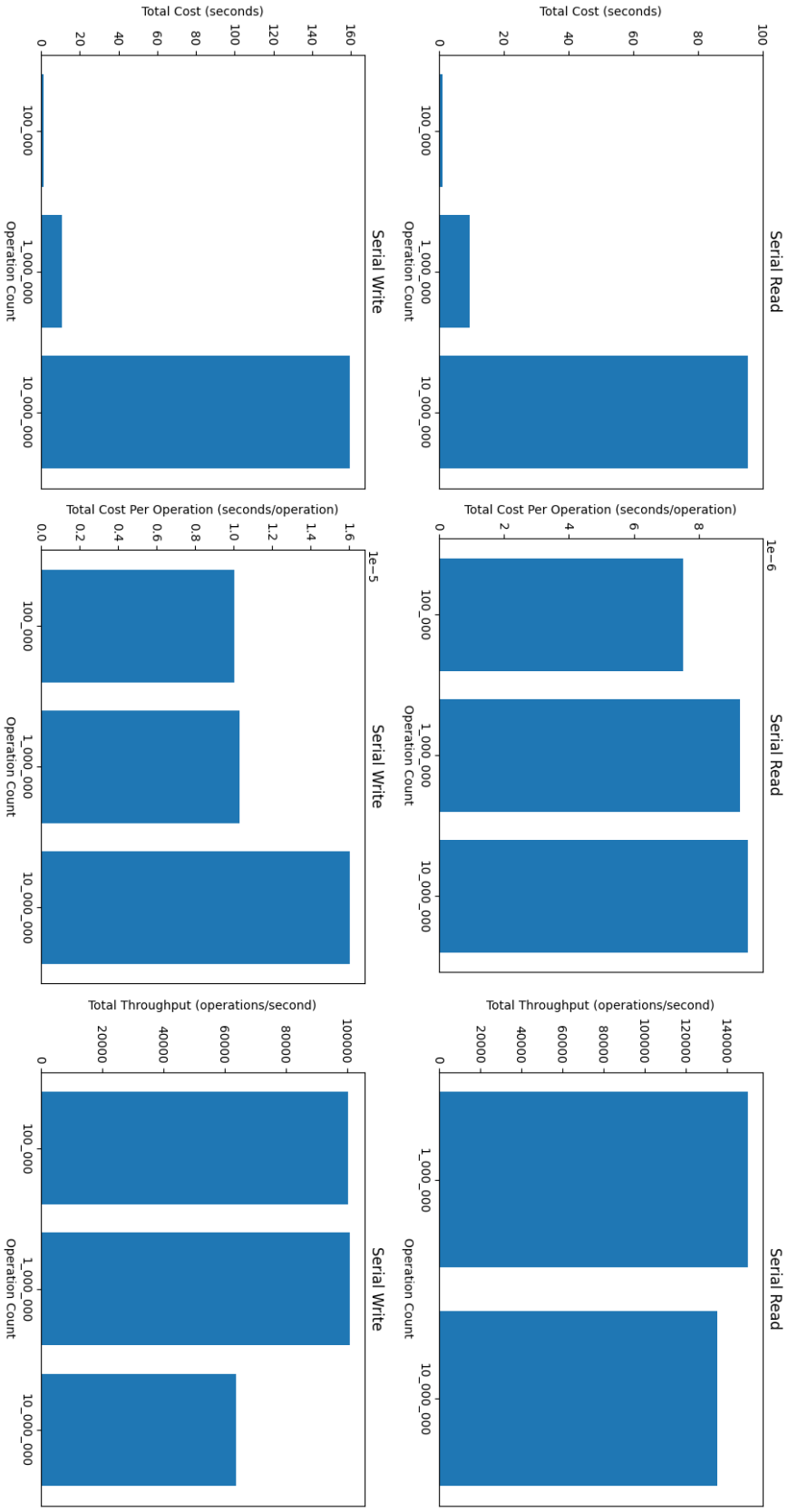
```
...
fprintf(stdout, "Total writer cost: %fsec, Total writer cost per operation:
↪   %fsec/op, Total writer throughput: %fops/sec\n",
                writers_total_cost, writers_total_cost_per_op,
↪   writers_total_throughput);
    fprintf(fstats, "%s, %ld, 1, 100, %d, %f, -1, %f, -1, %f, -1\n",
    argv[1], count, r, writers_total_cost, writers_total_cost_per_op,
↪   writers_total_throughput);
...
```

και τον παρακάτω κώδικα για το σειριακό read.

```
...
fprintf(stdout, "Total reader cost: %fsec, Total reader cost per operation:
↪   %fsec/op, Total reader throughput: %fops/sec\n",
                readers_total_cost, readers_total_cost_per_op,
↪   readers_total_throughput);
    fprintf(fstats, "%s, %ld, 1, 100, %d, %f, -1, %f, -1, %f, -1\n",
    argv[1], count, r, readers_total_cost, readers_total_cost_per_op,
↪   readers_total_throughput);
...
```

# Απόδοση Συστήματος

## Σειριακή Ανάγνωση/Εγγραφή




Σχήμα 1: Στατιστικά απόδοσης σειριακών λειτουργιών.

Στο σχήμα 1 παρατηρούμε τα στατιστικά απόδοσης των σειριακών λειτουργιών. Παρατηρούμε αύξηση των μετρικών του κόστους όσο αυξάνεται ο αριθμός των λειτουργιών που επιτελούνται στη βάση. Αυτό είναι ακόμα πιο εμφανές στη μετρική του συνολικού κόστους στη σειριακή εγγραφή. Στην περίπτωση της σειριακής εγγραφής αξίζει να σημειωθεί πως υπάρχει επιπρόσθετο χρονικό κόστος καθώς όσο αυξάνεται ο αριθμός των τιμών για εγγραφή, αυξάνεται και η πιθανότητα να χρειαστεί compaction των δομών αποθήκευσης. Επίσης στη διεκπεραιοτικότητα της σειριακής ανάγνωσης βλέπουμε αποτελέσματα μόνο για 1000000 και 10000000 στοιχεία καθώς τα 100000 στοιχεία είναι πολύ λίγα για να γίνει επαρκής μέτρηση της τιμής (Η βάση τα επεξεργάζεται σε κλάσματα του δευτερολέπτου ($\approx 0$) με αποτέλεσμα να έχουμε ως αποτέλεσμα άπειρο throughput).

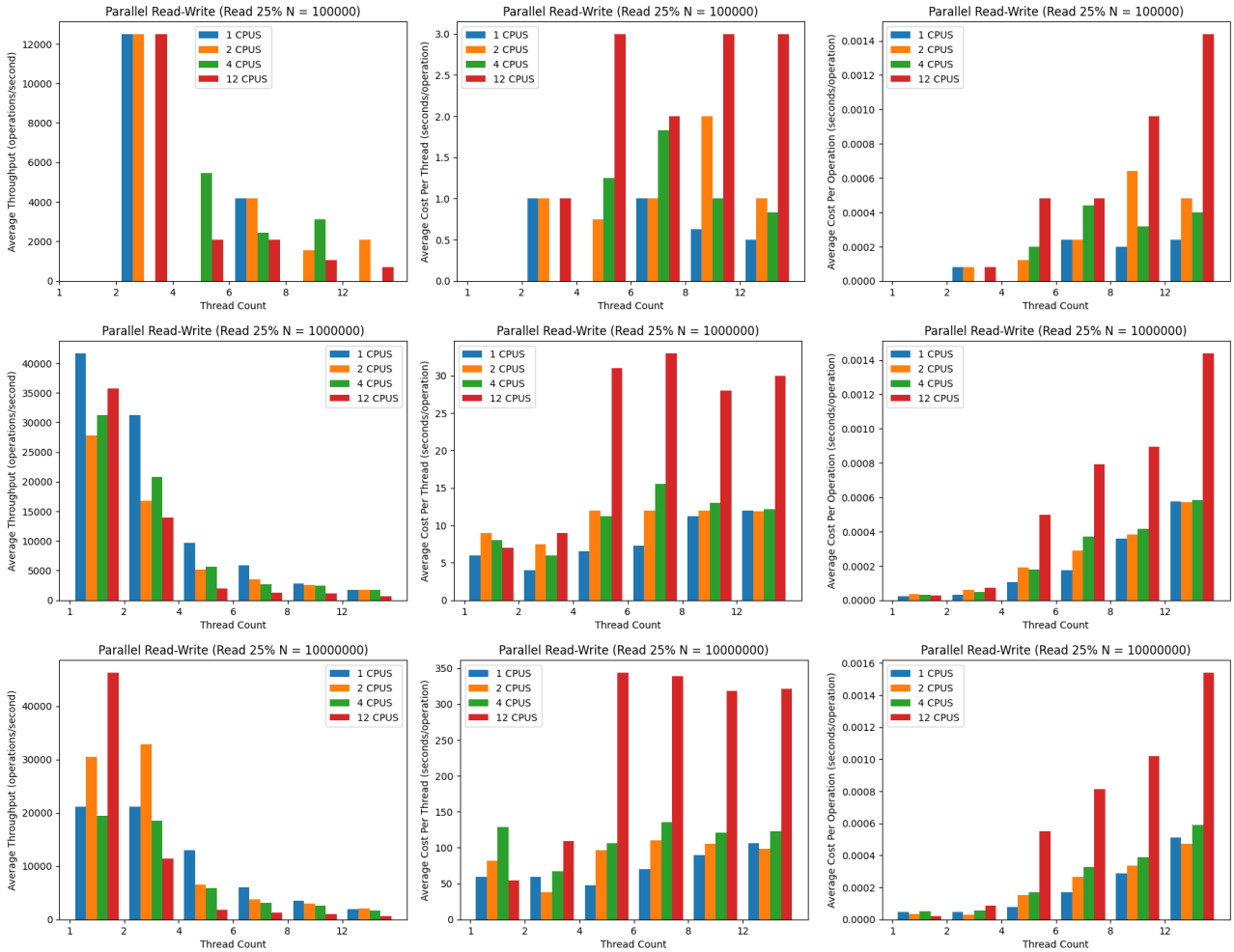
## Παράλληλη Ανάγνωση

Σχήμα 2: Στατιστικά απόδοσης παράλληλης ανάγνωσης.

Στο σχήμα 2 παρατηρούμε μια βελτίωση στο κόστος ανά λειτουργία κάτι το οποίο είναι φυσιολογικό λόγω της εφαρμογής παραλληλισμού στο πρόγραμμα. Βελτίωση παρατηρούμε επίσης στο μέσο κόστος ανά νήμα. Αντίθετα παρατηρούμε μείωση του μέσου throughput (λίγο παραπάνω από 50%) στα 2 νήματα ενώ μετά εξομαλύνεται και έχει ελαφρώς καλύτερη απόδοση (>4 νημάτων).

## Μίγμα Αναγνώσεων Εγγραφών

### 25% Αναγνώσεις

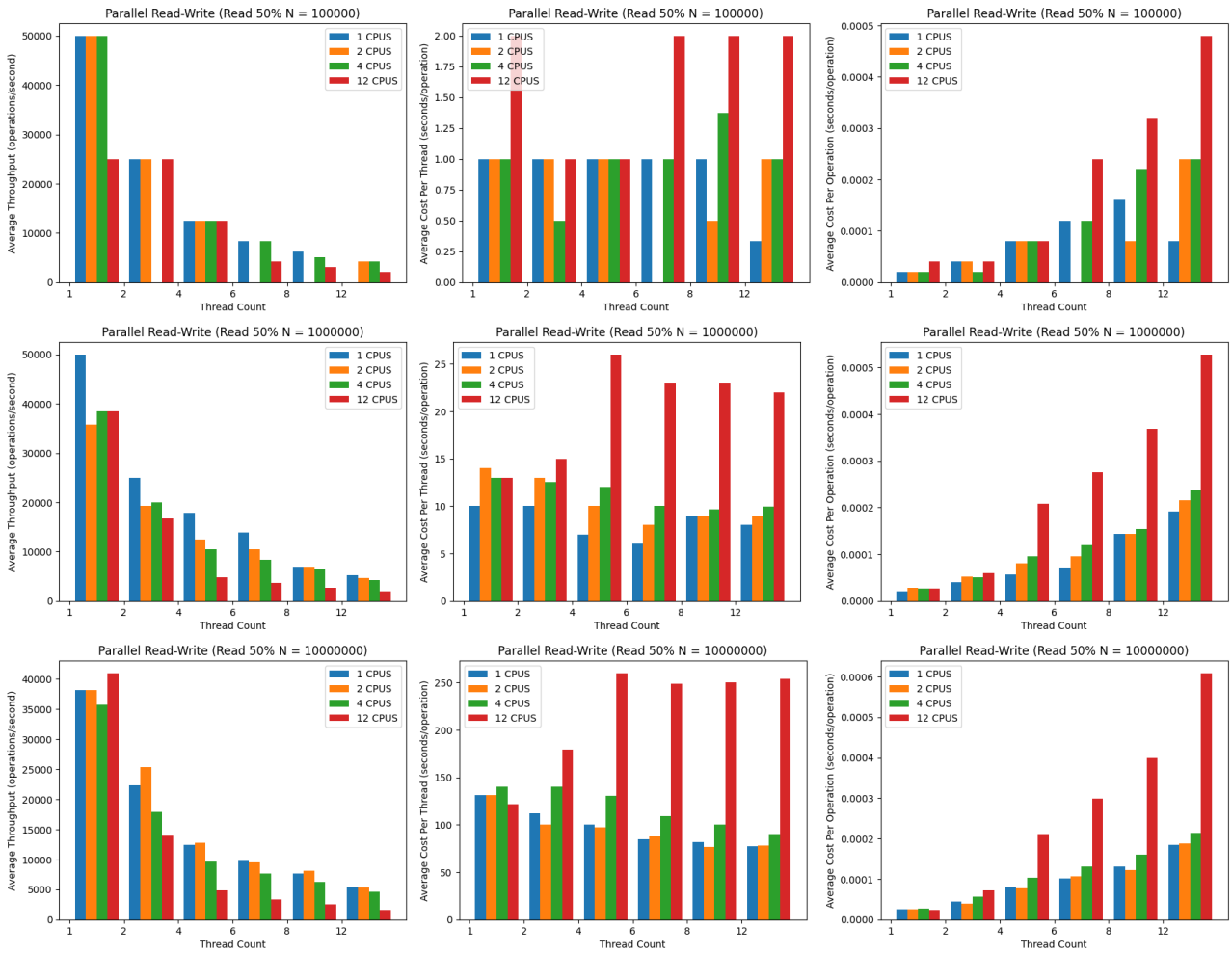



Σχήμα 3: Στατιστικά απόδοσης παράλληλης ανάγνωσης 25% από μίγμα αναγνώσεων/εγγραφών.

Στο σχήμα 3 παρατηρούμε πως το μέσο κόστος ανά λειτουργία παραμένει σταθερό ασχέτως του αριθμού των λειτουργιών που επιτελούμε αλλά αυξάνεται άν αυξήσουμε τον αριθμό των πυρήνων που αξιοποιεί η εικονική μηχανή. Το μέσο κόστος ανά νήμα επηρεάζεται σχεδόν ανάλογα με τον αριθμό των λειτουργιών που επιτελούμε ενώ η χρήση παραπάνω πυρήνων δεν επηρεάζει σημαντικά το κόστος (βλέπουμε βέβαια μια μεγάλη αύξηση σε αυτό όταν χρησιμοποιήσαμε 12 πυρήνες και έχουμε 6 ή περισσότερα νήματα να εργάζονται ταυτόχρονα). Όσον αφορά το μέσο throughput, παρατηρούμε μια μείωση όσο αυξάνουμε τον αριθμό των επιτελούντων λειτουργιών αλλά και όσο αυξάνουμε τον αριθμό των ταυτόχρονων νημάτων. Αυτό μπορεί να οφείλεται στο γεγονός πως οι αναγνώστες δίνουν προτεραιότητα στους γραφείς και πως σε περιπτώσεις οι δομές αποθήκευσης κάνουν compaction, κάτι το οποίο επηρεάζει αρκετά το χρόνο αναμονής των νημάτων.

### 50% Αναγνώσεις

Στο σχήμα 4 παρατηρούμε πως το μέσο throughput παραμένει σταθερό άσχετα με τον αριθμό των λειτουργιών που επιτελούμε (παρατηρούμε και εδώ μείωσή του όσο αυξάνουμε τον αριθμό των ταυτόχρονων νημάτων) αλλά παρατηρούμε μια σχεδόν εκθετική αύξηση



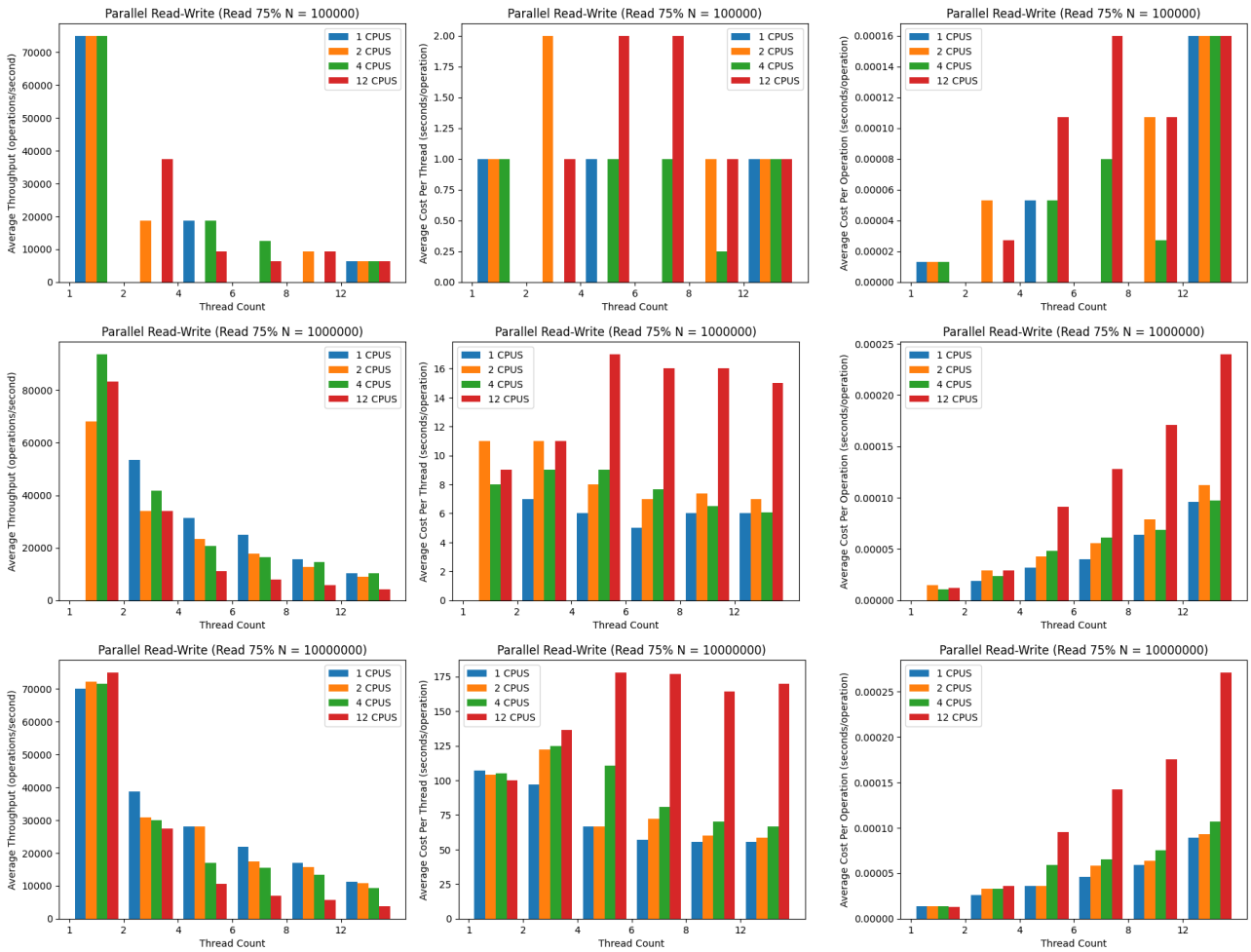

Σχήμα 4: Στατιστικά απόδοσης παράλληλης ανάγνωσης 50% από μίγμα αναγνώσεων/εγγραφών.

στο μέσο κόστος ανά νήμα. Το μέσο κόστος ανά λειτουργία φαίνεται να μειώνεται σε σχέση με το προηγούμενο σχήμα αλλά έχουμε και εδω αύξησή του όταν ανεβάζουμε τον αριθμό των ταυτόχρονων νημάτων. Και εδώ παρατηρούμε απότομες αλλαγές όταν χρησιμοποιούμε στην εικονική μηχανή 12 πυρήνες και πάνω από 6 ταυτόχρονα νήματα.

### 75% Αναγνώσεις

Στο σχήμα 5 παρατηρούμε παρόμοια αποτελέσματα με τα παραπάνω σχήματα αλλά με καλυτέρευση των στατιστικών κόστους αλλά και του throughput κάτι το οποίο οφείλεται κυρίως στη χρήση λιγότερων λειτουργιών εγγραφής άρα και λιγότερων προσπαθειών για compaction στις δομές αποθήκευσης.

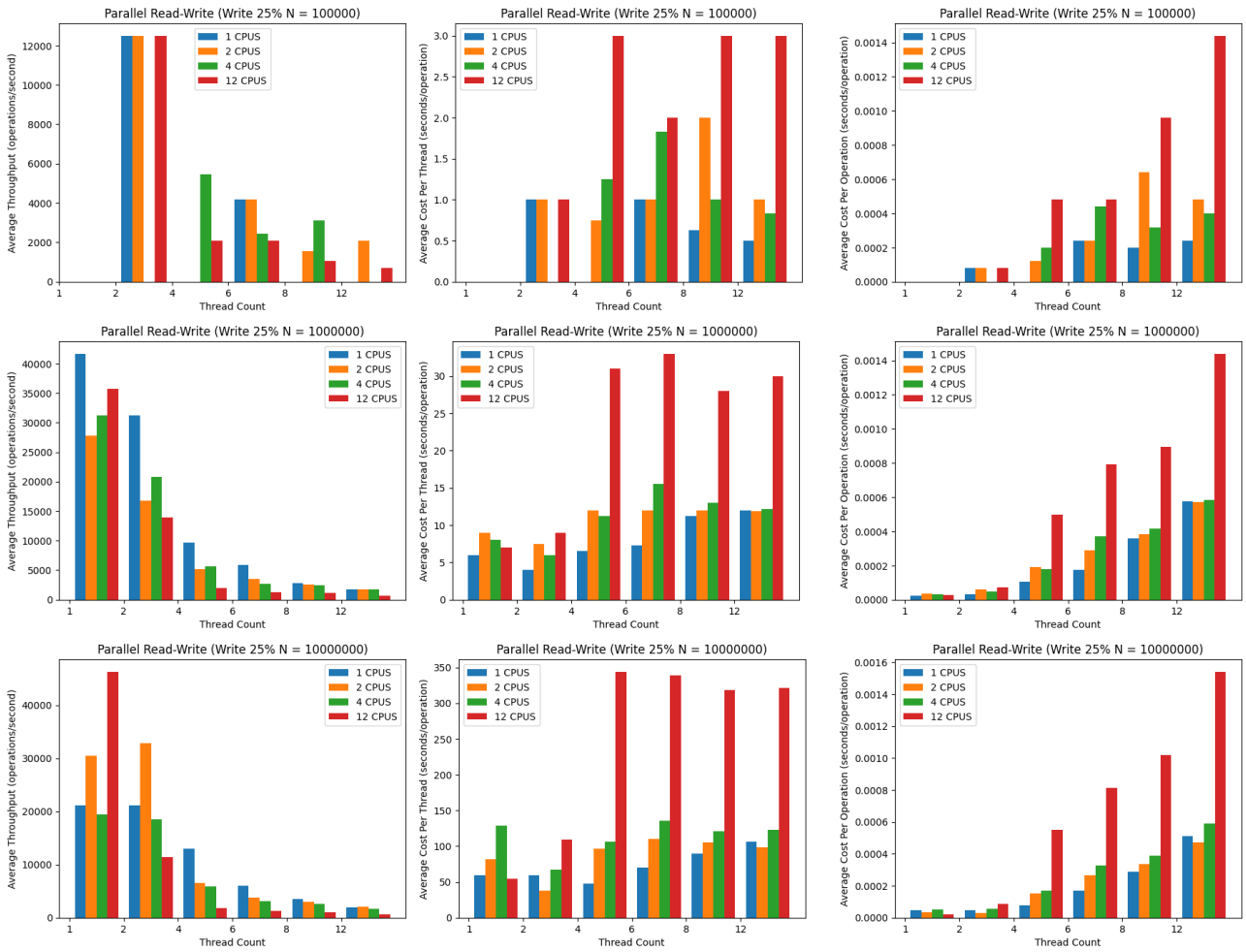
Στη λειτουργία μίγματος παράλληλων αναγνώσεων/εγγραφών η απόδοση των αναγνώσεων καλυτερεύει όσο μειώνουμε το ποσοστό των εγγραφών με την καλύτερη περίπτωση να είναι όταν έχουμε 75% αναγνώσεις και 25% εγγραφές.




Σχήμα 5: Στατιστικά απόδοσης παράλληλης ανάγνωσης 75% από μίγμα αναγνώσεων/εγγραφών.

## 25% Εγγραφές

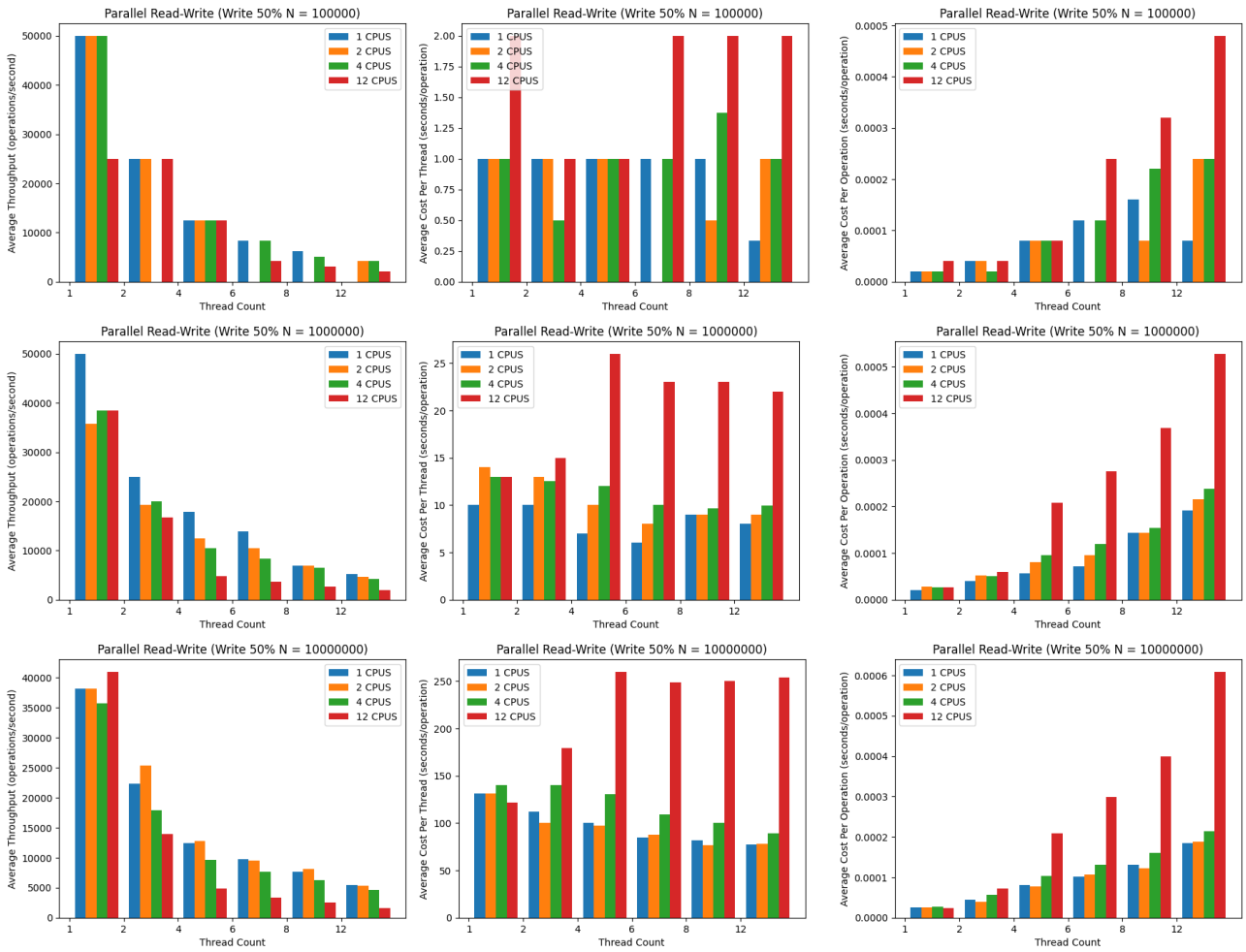
Στο σχήμα 6 παρατηρούμε βελτίωση του throughput με την αύξηση του αριθμού των λειτουργιών, εκτός αν ο αριθμός τους είναι πραγματικά πολύ μεγάλος, όπως στην περίπτωση των 10.000.000. Επιπλέον, παρατηρούμε πως με την αύξηση του αριθμού των νημάτων έχουμε βελτίωση αρχικά για μικρή ποσότητα νημάτων, αλλά καθώς ο αριθμός των νημάτων αυξάνεται, ο επιπλέον φόρτος ταυτοχρονισμού των διεργασιών χειροτερεύει την επίδοση του προγράμματος. Ο αριθμός επεξεργαστών φαίνεται να επηρεάζει αρκετά τις επιδόσεις του προγράμματος καθώς για 2 επεξεργαστές έχουμε γενικότερα καλύτερες επιδόσεις έναντι του 1, ενώ παρατηρούμε ότι με 4 επιστρέφουμε σε παρόμοιες επιδόσεις με εκτέλεση σε 1 επεξεργαστή. Στην περίπτωση των 12 επεξεργαστών παρατηρουμε σημαντική βελτίωση του throughput για χαμηλό αριθμό νημάτων, ειδικά όταν είναι υψηλός και ο αριθμός λειτουργιών αλλά ραγδαία μείωση επιδόσεων όσο ο αριθμός νημάτων αυξάνεται.




Σχήμα 6: Στατιστικά απόδοσης παράλληλης εγγραφής 25% από μίγμα αναγνώσεων/εγγραφών.

## 50% Εγγραφές

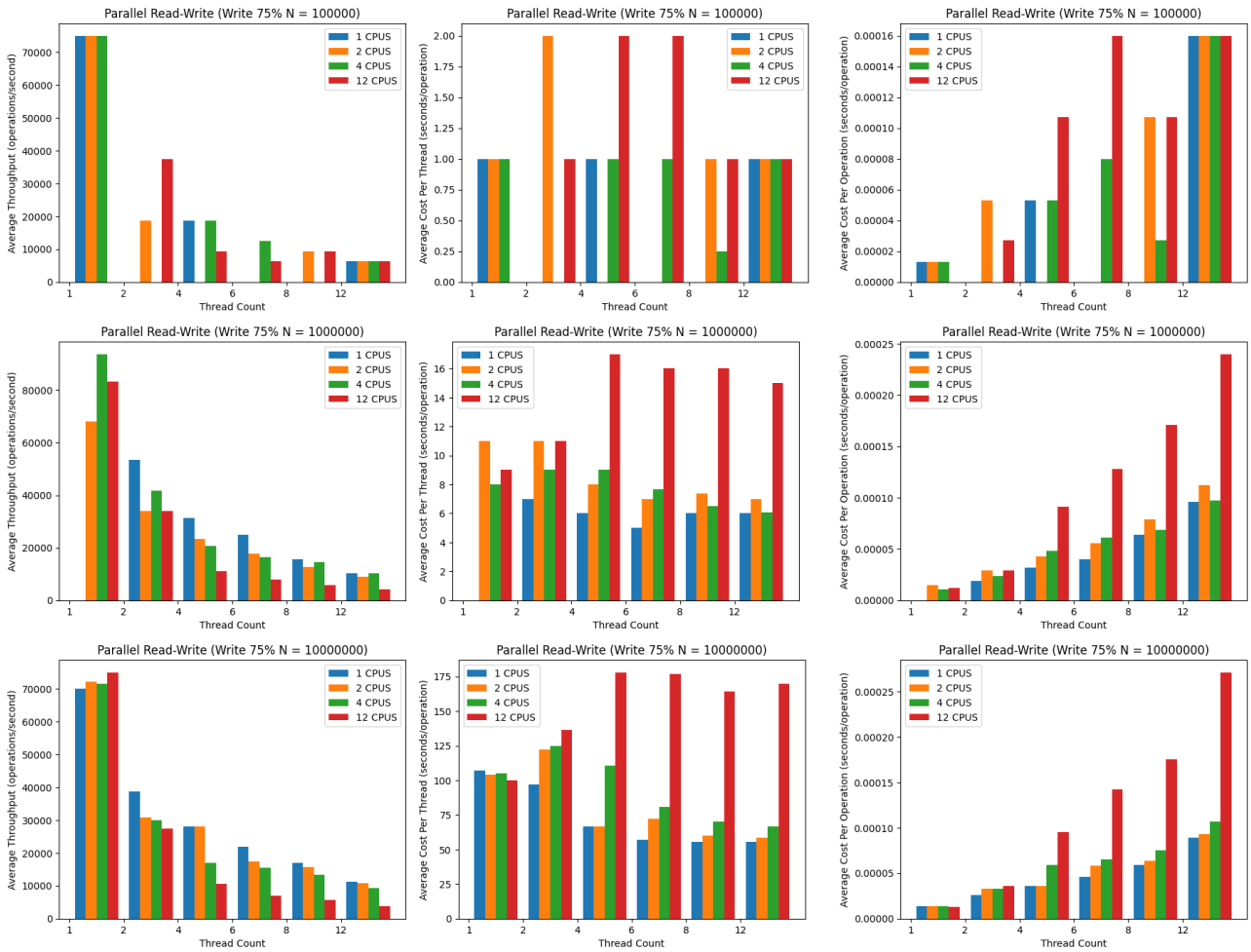
Στο σχήμα 7 παρατηρούμε πως το κόστος αυξάνεται ανάλογα με την αύξηση των λειτουργιών, κρατώντας το throughput σχετικά σταθερό. Η αύξηση των νημάτων προκαλεί βελτίωση στην λειτουργία του προγράμματος εφόσον έχουμε πολύ χαμηλές αυξήσεις στο κόστος ανά λειτουργία και μειώσεις στο κόστος ανά νήμα, διατηρώντας τον μέσο όρο throughput ανά νήμα αρκετά υψηλά ώστε να γίνεται η εκτέλεση των λειτουργιών συνολικά γρηγορότερα. Παρ'όλα αυτά, η άυξηση των νημάτων έχει diminishing returns, όπως και είναι φυσιολογικό για ένα πρόγραμμα που δεν τρέχει 100% παράλληλα, συνεπώς έχουμε όλο και μικρότερη βελτιώση για κάθε αύξηση στον αριθμό νημάτων. Ο αριθμός επεξεργαστών αρχικά προκαλεί βέλτιωση στην λειτουργία του προγράμματος αλλά αυξάνει δραματικά το κόστος εκτέλεσης των λειτουργιών, ειδικά σε συνδυασμό με υψηλό αριθμό νημάτων, για μεγάλο αριθμό επεξεργαστών, όπως βλέπουμε πως συμβάινει για 12 επεξεργαστές, λόγω του επιπλέον ταυτοχρονισμού που χρειάζεται.




Σχήμα 7: Στατιστικά απόδοσης παράλληλης εγγραφής 50% από μίγμα αναγνώσεων/εγγραφών.

## 75% Εγγραφές

Στο σχήμα 8 παρατηρούμε παρόμοια συμπεριφορά στην αλλαγή των επιδώσεων με την περίπτωση των 50% γραφέων. Πιό συγκεκριμένα, έχουμε και πάλι ανάλογη αύξηση του κόστους με την αύξηση του αριθμού των λειτουργιών, ενώ το throughput παραμένει σχετικά σταθερό. Η αύξηση των νημάτων επιφέρει βελτίωση του κόστους ανά thread με ελαφριά επιβάρυνση του κόστους ανά λειτουργία, χωρίς όμως να μειώνεται πολύ το μέσο throughput, επιφέροντας συνολικά βελτίωση στη λειτουργία του προγράμματος. Τέλος, παρατηρούμε πως ο αριθμός επεξεργαστών επιφέρει βέλτιωση στη λειτουργία του προγράμματος αλλά αυξάνει δραματικά το κόστος εκτέλεσης λειτουργιών, ειδικά σε συνδυασμό με υψηλό αριθμό νημάτων, για μεγάλο αριθμό επεξεργαστών, όπως βλέπουμε πως συμβάινει για 12 επεξεργαστών, λόγω του επιπλέον ταυτοχρονισμού που χρειάζεται. Επιπλέον, εξαίρεση στην παρπάνω παρατήρηση αποτελεί η περίπτωση εκτέλεσης του προγράμματος με 2 νήματα η οποία φαίνεται να επιφέρει μείωση της απόδοσης με την αύξηση του αριθμού των επεξεργαστών.




Σχήμα 8: Στατιστικά απόδοσης παράλληλης εγγραφής 75% από μίγμα αναγνώσεων/εγγραφών.

# Γενικές παρατηρήσεις

Είναι πιθανόν να υπάρχουν λογικά λάθη και ελλείψεις στο κομμάτι του κώδικα παρ' όλα αυτά η άσκηση εκτελείται κατά κύριο λόγο χωρίς προβλήματα. Αποφασίσαμε να αφιερώσουμε αρκετό χρόνο στην αξιολόγηση του συστήματος καθώς αυτή είναι πολύ σημαντικό κομμάτι στην παραλληλοποίηση και στην εκτίμηση της ορθότητας ενός λογισμικού. Θεωρούμε πως έχουμε παραδώσει μια αξιόλογη προσπάθεια.

Σας ευχαριστούμε για το χρόνο σας!